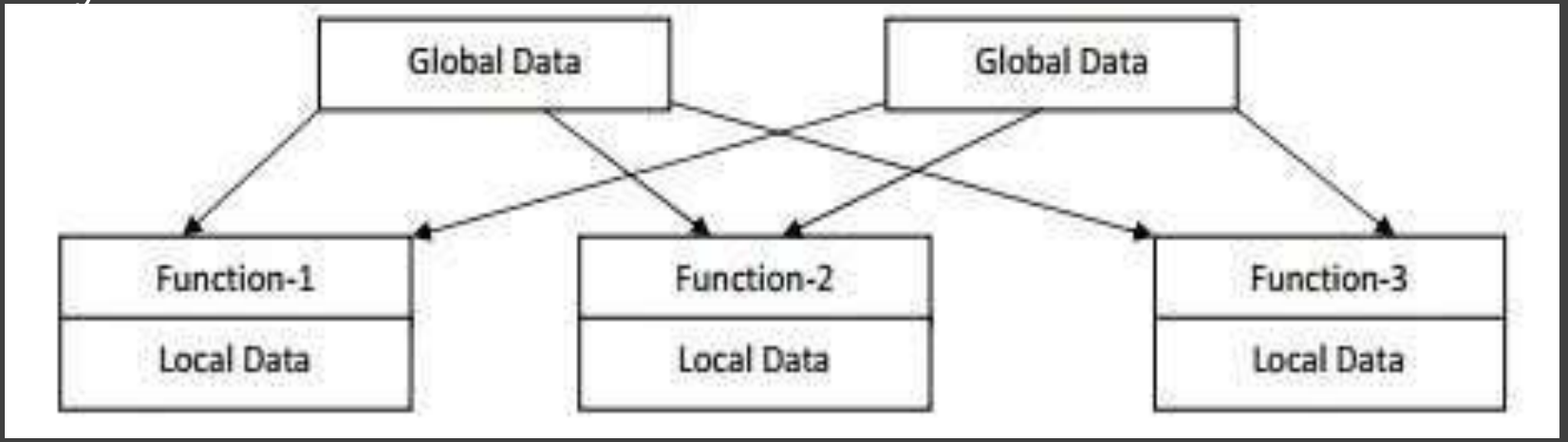
Introduction to Object Oriented Programming

Procedure/ Structure Oriented Programming

- Conventional programming, using high level languages such as COBOL, FORTRAN and C, is commonly known as procedure- oriented programming (POP) .
- In the procedure-oriented approach, the problem is viewed as a sequence of things to be done such as reading, calculating and printing. A number of functions are written to accomplish these tasks .
- The primary focus is on functions :

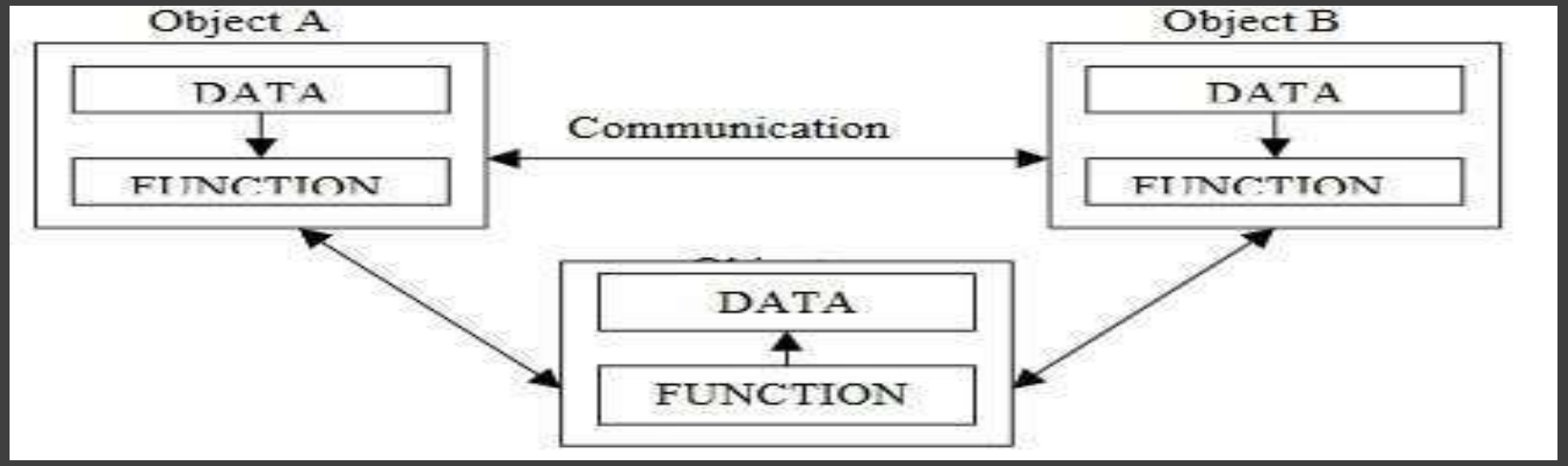


Object Oriented Paradigm

- Programming paradigm is a style or approach to writing program.
- The **Object-Oriented (OO) Paradigm** is a programming methodology in which a program is organized around **classes** and **objects** rather than functions.
- An **object** represents a **real-world entity** (e.g., Student, Car, Bank Account).
- Class is a blueprint of these entities that defines the **data members (attributes)** and **member functions (methods)**

Object Oriented Programming

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- *Follows bottom-up* approach in program design



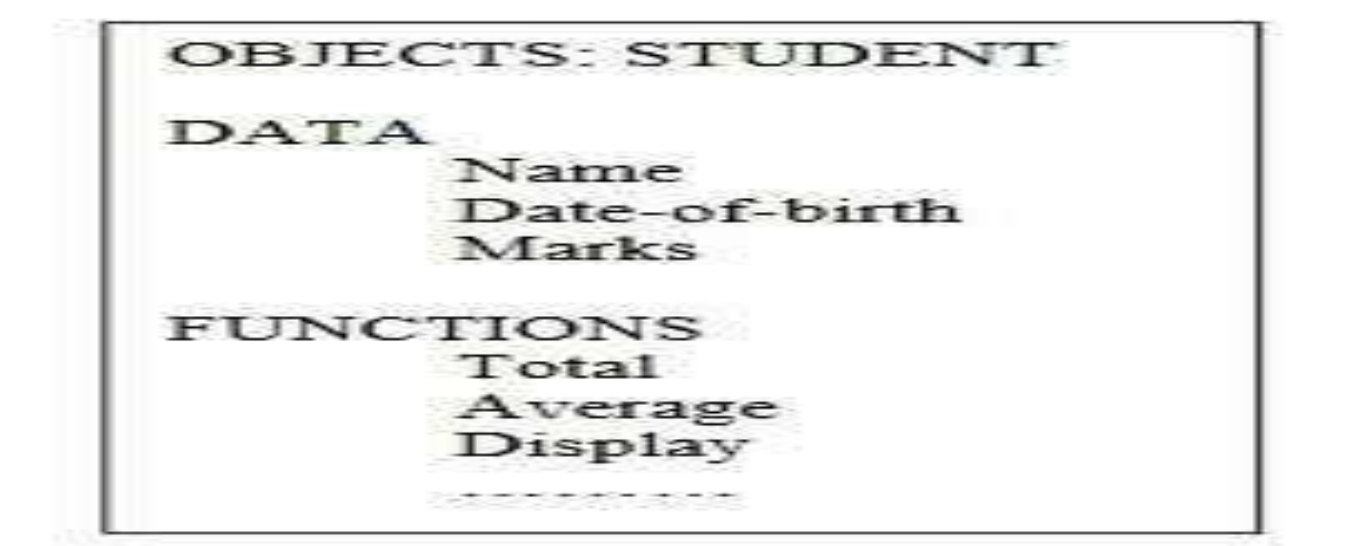
Object Oriented Paradigm

- It promotes:
  - **Modularity**
  - **Code Reusability**
  - **Maintainability**
  - **Scalability**
  - **Data Security**
- OOP is widely used for developing **large and complex software systems**

Basic Concepts of OOP

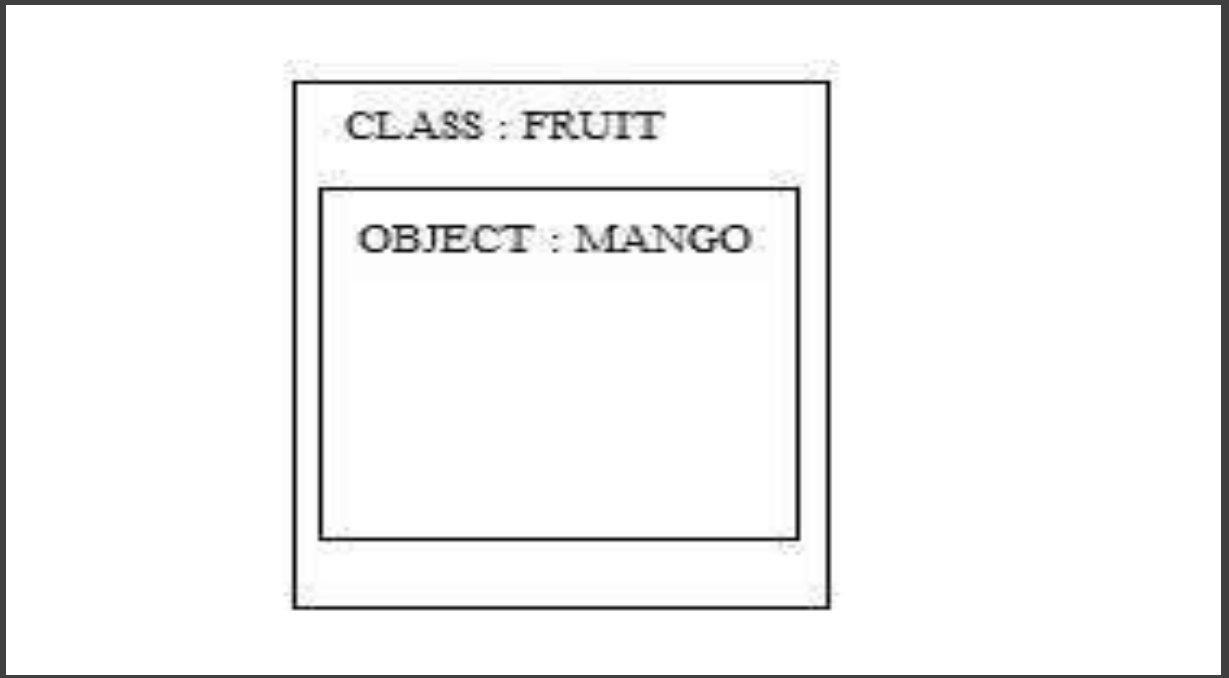
➢Objects :

Objects are the basic runtime entities in an object oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program has to handle



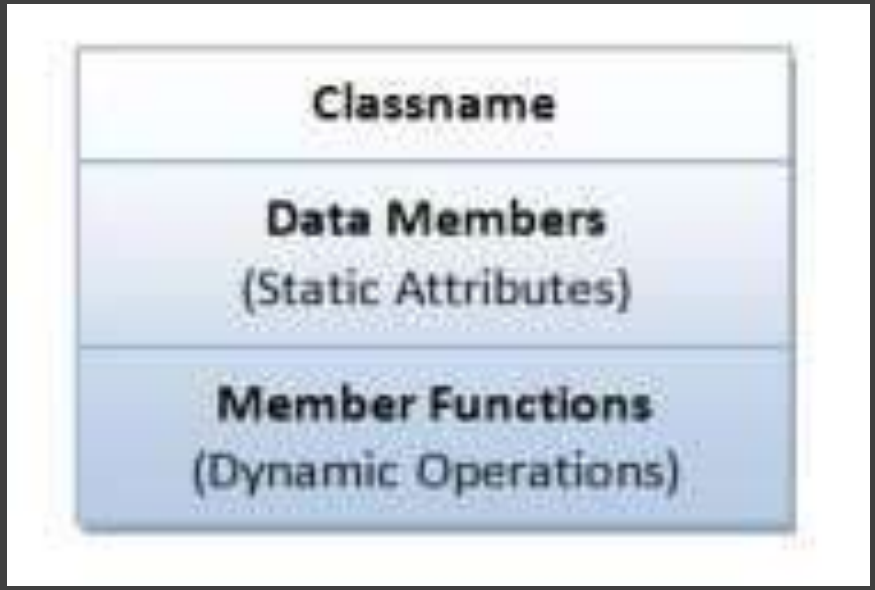
Class

Object contains data, and code to manipulate that data. The entire set of data and code of an object can be made a user-defined data type with the help of a class



- A Class is a 3-Compartment Box encapsulating Data and Functions

1. **Classname** (or identifier): identifies the class.
2. **Data Members** or Variables (or attributes, states, fields): contains the static attributes of the class.
3. **Member Function (**or methods, behaviors, operations): contain the dynamic operations of the class.



Data Encapsulation

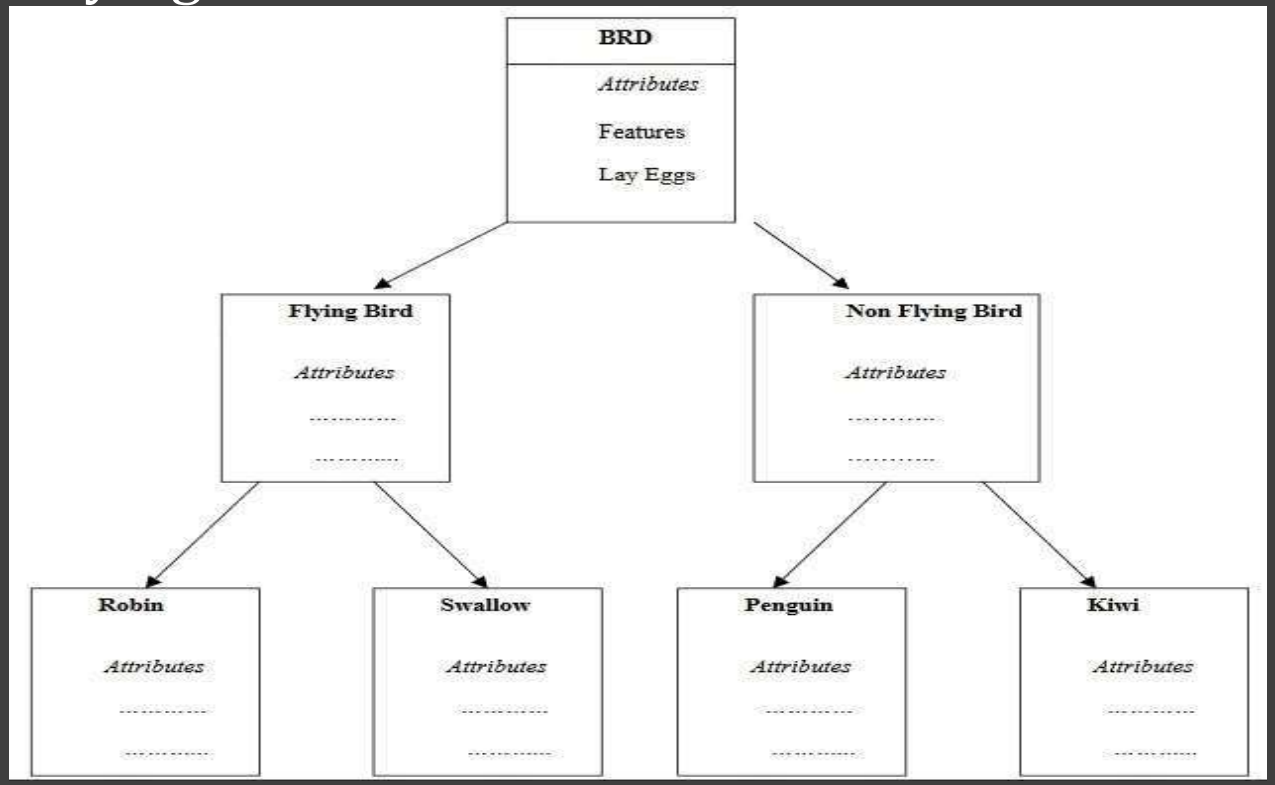
- The wrapping up of data and functions into a single unit is known as encapsulation.
- The data is not accessible to the outside world, only those function which are wrapped in the can access it.
- These functions provide the interface between the object's data and the program.
- This insulation of the data from direct access by the program is called data hiding or information hiding.

Data Abstraction

- Abstraction refers to the act of representing essential features without including the background details or explanations.
- Since classes use the concept of data abstraction, they are known as **Abstract Data Types (ADT)**

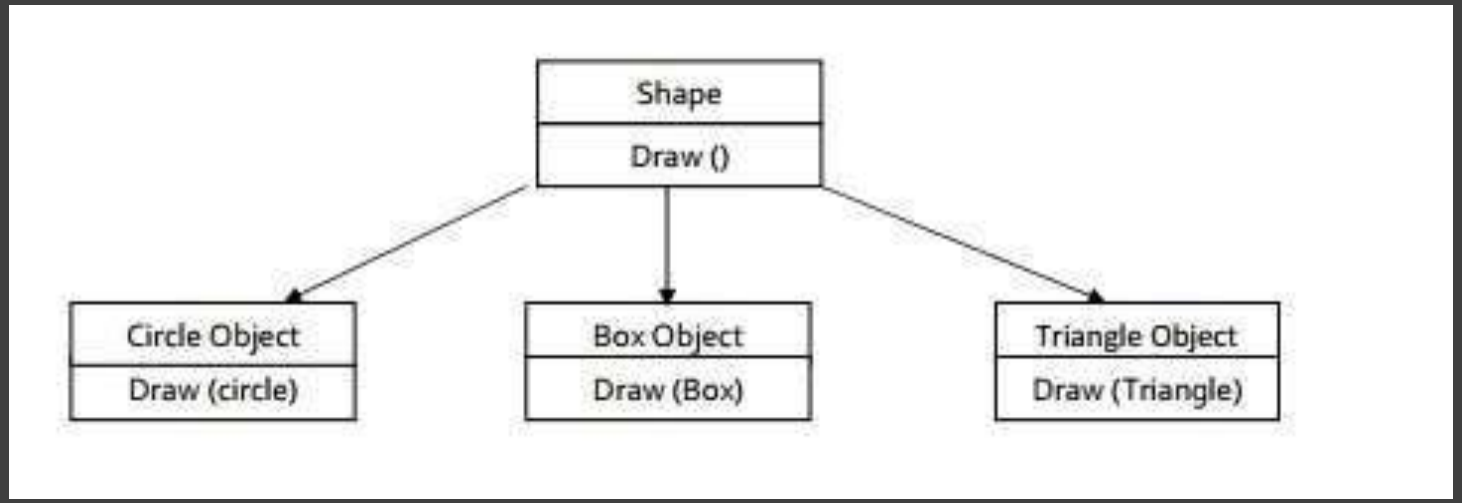
Inheritance

- Inheritance is the process by which objects of one class acquire the properties of objects of another class.
- In OOP, the concept of inheritance provides the idea of reusability. This means we can add additional features to an existing class without modifying it.



Polymorphism

- **Polymorphism,** a Greek term means to ability to take more than one form.
- An operation may exhibits different behaviors in different instances. The behavior depends upon the type of data used in the operation.
- For example consider the operation of addition for two numbers; the operation will generate a sum. If the operands are string then the operation would produce a third string by concatenation.
- The process of making an operator to exhibit different behavior in different instances is known operator overloading



Benefits of OOP

OOP offers several benefits to both the program designer & the user

- Through inheritance, we can eliminate redundant code and extend the use of existing class
- We can build programs from the standard working module the communicate with one another, rather than having to start writing code from scratch. This leads to saving of development time & higher productivity.
- The principle of data hiding helps the programmer to build & secure programs that cannot be invaded by code in other parts of the program.
- It is possible to map objects in the problem domain to those in the program
- It is easy to partition the work in a project based on objects
- The data-centered design approach enables us to capture more details of a model in implementable form.

Benefits of OOP

- Object-oriented systems can be easily upgraded from small to large systems.
- Message passing techniques for communication between objects makes the interface descriptions with external systems much simpler.
- Software complexity can be easily managed .

Application of OOP

Real-business systems are often much more complex and contain many more objects with complicated attributes and methods. OOP is useful in these types of applications because it can simplify a complex problem. The promising areas for application of OOP include:

- Real-time systems
- Simulation and modeling
- Object-oriented databases
- Hypertext, hypermedia and expertext
- Al and expert systems
- Neural networks and parallel programming
- Decision support and office automation systems
- CIM/CAM/CAD systems

Structured Vs OOP

Aspect	Structured Programming	Object-Oriented Programming (OOP)
Basic Concept	Programs are developed by dividing the problem into functions, where each function performs a specific task.	Programs are developed using classes and objects, which represent real-world entities and their behavior.
Programming Approach	Uses a Top-Down approach, starting with the main problem and breaking it into smaller functions.	Uses a Bottom-Up approach, where individual objects are created first and then combined to build the complete application.
Data and Functions	Data and functions are separate, so multiple functions can directly access and modify the data.	Data and methods are combined into a single unit (object), allowing controlled

Structured Vs OOP

Aspect	Structured Programming	Object-Oriented Programming (OOP)
Security and Reusability	Provides less data security and offers limited code reusability.	Provides better data security through encapsulation and promotes code reusability through inheritance.
Maintenance	As the program grows, it becomes difficult to maintain, debug, and modify.	The modular design makes programs easier to maintain, test, and extend.
Applications	Best suited for small and simple applications with limited complexity.	Ideal for large, complex, and real-world applications where scalability and maintainability are important.
Examples	C, Pascal	C++, Java, Python

Disadvantages of OOP

Higher Memory and Performance Overhead

- OOP programs generally require **more memory** and may execute slightly slower than procedural programs due to the use of objects and additional features.

Not Suitable for Small Applications

- For simple or small programs, using OOP may introduce unnecessary complexity and increase development effort.

Complex Design

- Designing classes, objects, and their relationships requires careful planning and good programming skills.
- Poor design can make the application difficult to manage.

Disadvantages of OOP

Steep Learning Curve

- Beginners need time to understand OOP concepts such as **classes, objects, inheritance, polymorphism, abstraction, and encapsulation**.

Increased Complexity

- Treating every entity as an object can sometimes make the program structure more complicated than necessary, especially for straightforward problems.

C++ Syntax

```
#include <iostream>      // Header file
using namespace std;     // Use standard namespace

int main()               // Main function
{
        cout << "Hello, World!";   // Output statement

        return 0;                  // End of program
}
```

Class Specification

- A **class** is a user-defined data type in C++ that combines **data members** and **member functions** into a single unit.
- Syntax:

```
class ClassName
{
private:
    // Data members

public:
    // Member functions
};
```

Class Objects

- An **object** is an instance of a class. It is used to access the data members and member functions of the class.
- Syntax:

  ClassName objectName;
- We can create multiple objects for one class.

Member Function

- The functions declared inside the class are knows as Member Function.
- Member functions access the values of the data members and perform operations on the data members.
- The definition of member functions can be inside or outside the class.
- Can access both public and private member of the class.

Defining Member Function

- Member function can be defined in two places:
  - Outside the class definition
  - Inside the class definition

Inside the class Definition

- Replace the function declaration by the actual definition inside the class.
- When a function is defined inside a class it is treated as inline function.

Defining Member Function

- All the restrictions and limitations that apply to an inline function are also applicable here.
- Syntax :

```
ret_type fun_name(formal parameters)
 {
function body;
}
```

Defining Member Function

```
class Item
{
   int number;
   float cost;

public:
   void getdata(int a, float b);   // Function declaration

   void putdata(void)            // Function definition inside class
   {
      cout << number << "\n";
      cout << cost << "\n";
   }
};
```

Defining Member Function

Outside the Class Definition

- Member functions that are declared inside a class have to be defined separately outside the class.
- Their definitions are very much like the normal functions.
- They should have a function header and a function body.
- An important difference between a member function and a normal function is that a member function incorporates a membership 'identity label' in the header.
- This 'label' tells the compiler which class the function belongs to.

Defining Member Function

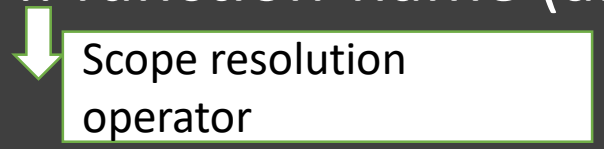
- The general form of a member function definition is:

return-type class-name :: function-name (argument declaration)

{

Function body

}



Defining Member Function

```
#include <iostream>
using namespace std;

class Item
{
   int number;
   float cost;

public:
   void getdata(int a, float b);   // Function declaration
};

void Item::getdata(int a, float b)  // Function definition outside class
{
   number = a;
   cost = b;
}
```

Program using member function

```
#include <iostream>
using namespace std;

class item
{
   int number;      // private by default
   float cost;      // private by default

public:
   void getdata(int a, float b);   // function declaration

   void putdata(void)            // function defined inside class
   {
      cout << "Number: " << number << "\n";
      cout << "Cost: " << cost << "\n";
   }
};
// Member function definition outside the class
void item::getdata(int a, float b)
{
   number = a;      // assigning value to private variable
   cost = b;       // assigning value to private variable
}
// Main program
int main()
{
   item x;         // create object x

   cout << "\nObject x\n";
   x.getdata(100, 299.95);
   x.putdata();

   item y;         // create object y

   cout << "\nObject y\n";

   y.getdata(200, 175.50);
   y.putdata();
   return 0;
}
```

.

Making a outside function inline

A member function outside the class definition and still make it inline by just using the **qualifier inline** in the header line of function definition.

Syntax:

```
inline return_type ClassName::FunctionName(arguments)
{
   // function body
}
```

.

Making a outside function inline

```
class item
{
public:
    void getdata(int a, float b);   // declaration
};

inline void item::getdata(int a, float b)   // definition
{
    number = a;
    cost = b;
}
```

.

inline function

- An inline function is a function in C++ whose code is expanded at the point of function call. It reduces the overhead associated with function calls and can improve execution speed.

•The **inline keyword** requests the compiler to replace the function call with the function's code.
•**Inlining is a request, not a command**; the compiler may ignore the request.
•Inline functions are generally suitable for **small functions**.
•The compiler may not perform inline expansion for functions that are too complex(like loops, control statements, recursion).

.

inline function

- An **inline function** is a function in which the compiler is requested to replace the function call with the actual function body.

Syntax:

```
inline return_type function_name(parameters)
{
  // function body
}
```

```
#include <iostream>
using namespace std;

inline float square(float x)
{
    return x * x;
}

int main()
{
    float n;

    cout << "Enter a number: ";
    cin >> n;

    cout << "Square = " << square(n);

    return 0;
}
```

Access Specifiers

Access specifiers define who can access the data members and member functions of a class.

Types:

1.private: accessed by only member functions of the class.

2.public : accessed by member functions and all other non-member functions in the program.

3.protected: similar to private, but accessed by all member functions of immediate derived class.

- Be default all items defined in the class are private.

Data hiding

- **Data hiding** means keeping the data of a class hidden so that it **cannot be accessed or modified directly by functions outside the class**.
- Data hiding is mainly achieved by declaring data members as **private.**

Empty class

- An empty class is a class that contains no data members and no member functions.
- Syntax:

```
class Empty
{
};
```

Why are Empty Classes Used?

- During the **initial stages of software development**, some classes may be identified but their actual implementation may not yet be decided.
- In such situations, an empty class can be created as a **placeholder**.
- These are often called **stubs** in this development context.
- Later data members and member functions can be added.

Passing Object as Parameter

- Like any other variable, objects can also be passed to the function, as an argument
- There are two ways of doing this
  - Pass by value
  - Pass by reference
- In the pass by value, the copy of object is passed to the function.
- So change made to the object inside the function do not affect the actual object.

Passing Object as Parameter

- On the other hand, address of the object is passed in the case of pass by reference
- So the changes made to the object inside the function are reflected in the actual object .
- This method is considered more efficient.
- Syntax:

```
function_name(object_name);
```

Passing Object by Value

a copy of the entire object is passed to the function

```
#include <iostream>
using namespace std;

class item
{
   int number;
   float cost;

public:
   void getdata(int a, float b)
   {
      number = a;
      cost = b;
   }
void putdata()
   {
      cout << "Number: " << number << endl;
      cout << "Cost: " << cost << endl;
   }
 void compare(item x)
   {
      if (cost > x.cost)
            cout << "This item is more expensive." << endl;
         else
            cout << "This item is less expensive." << endl;
   }
};

int main()
{
   item x, y;

   x.getdata(100, 299.95);
   y.getdata(200, 175.50);

   x.putdata();
   y.putdata();

   x.compare(y);       // object y passed as argument

   return 0;
}
```

Passing Objects by Reference

- only the address of the object is passed implicitly to the function

Syntax:

```
return_type function_name(ClassName &object_name);
```

Passing Objects by Reference

```
class item
{
   int number;
   float cost;

public:

   void getdata(int a, float b)
   {
      number = a;
      cost = b;
   }

   void change(item &x)
   {
      x.cost = 500;
   }
void putdata()
   {
      cout << "Number: " << number << endl;
      cout << "Cost: " << cost << endl;
   }
};
```

```
int main()
{
   item x;

   x.getdata(100, 299.95);

   cout << "Before change:" << endl;
   x.putdata();

   x.change(x);

   cout << "\nAfter change:" << endl;
   x.putdata();

   return 0;
}
```

Returning Objects from Member Function

- The method of returning an object from member function is same as returning a simple variable.
- If a member function returns an object, its return type should be the same as the type of object to be returned.

Syntax

```
ClassName functionName(arguments)
{
   ClassName object;

   // operations

   return object;
}
```

Returning Objects from Member Function

```
class Number
{
   int n;

public:
   void getdata(int x)
   {
      n = x;
   }

   Number add(Number x)
   {
      Number temp;

      temp.n = n + x.n;

      return temp;
   }
   void display()
   {
      cout << "Sum = " << n << endl;
   }
};

int main()
{
   Number n1, n2, n3;

   n1.getdata(10);
   n2.getdata(20);

   n3 = n1.add(n2);

   n3.display();

   return 0;
}
```

Static data and member functions

- **A static data member** has properties similar to a static variable declared in C.
- The important feature is that **only one copy of the static data member exists for the entire class**, and all objects share that copy.
- A static data member is declared using the keyword static.
- It is visible only within the class, but its lifetime is the entire program.
- Static variables are normally used to maintain values common to the entire class.

Static Data Member

- Syntax:
  - static data_type data_member_name; // declaration
  - type class name : : static datamember; // definition
- Eg: int item :: count;
- Static member variable must be defined outside the class definition. They are also know as class variable.

static data member

```
class item
{
public:
   static int count;   // static data member
   int number;

   void getdata(int a)
   {
      number = a;
      count++;
   }

   void getcount(void)
   {
      cout << "count: " << count << "\n";
   }
};
```

```
// Definition and initialization of static member
int item::count = 0;

int main()
{
   cout << "count: " << item::count << "\n";
   item a, b, c;

   a.getcount();
   b.getcount();
   c.getcount();

   a.getdata(100);
   b.getdata(200);
   c.getdata(300);

   cout << "After reading data" << "\n";

   a.getcount();
   b.getcount();
   c.getcount();

   return 0;
}
```

Static data and member functions

- Like static member variable, we can also have static member functions.

Static Member Function:

- **A member function** that is declared static has the following properties:
  - A static function can have access to only other static members (functions or variables) declared in the same class.
  - A static member function can be called using the-class name (instead of its objects) as follows:

    class-name :: function-name;

static member functions

```
class test
{
   int code;
   static int count;      // static member variable
public:
  void setcode(void)
   {
      code = ++count;
   }

   void showcode(void)
   {
      cout << "Object number: " << code << "\n";
   }

   static void showcount(void)   // static member function
   {
      cout << "Count: " << count << "\n";
   }
};
int test::count = 0;      // definition of static member
```

```
int main()
{
   test t1, t2;

   t1.setcode();
   t2.setcode();

   test::showcount();    // accessing static function

   test t3;

   t3.setcode();

   test::showcount();    // accessing static function

   t1.showcode();
   t2.showcode();
   t3.showcode();
   return 0;
}
```

Constructor

- A Constructor is a special member function that is called automatically when an object is created.
- The purpose of a constructor is to mainly initialize the member variables of a class.
- They should be declared in the public section.
- They do not have return types, not even void and therefore they cannot return values.
- They cannot be inherited, though a derived class can call the base class constructor. Like other C++ functions, they can have default arguments.

Constructor

Syntax:

```
class ClassName
{
   // private members

   ClassName();      // Constructor prototype

public:

   // public members
};

ClassName::ClassName()          // constructor definition
{
   // constructor body
}
```

Constructor

- Types of Constructor
  - Default constructor
  - Parameterized constructor
  - Copy constructor

Constructor

- Default constructor:

A default constructor is a constructor that can be invoked without supplying arguments. It may be explicitly defined by the programmer, or an appropriate default constructor may be implicitly generated by the compiler.

Constructor

- Default constructor:

```
class Student
{
    int rollNo;

public:
    Student()
    {
        rollNo = 101;
    }
};
```

Default Constructor

```
#include <iostream>
using namespace std;

class Student
{
   int rollNo;
   float marks;

public:

   // Default constructor
   Student()
   {
      rollNo = 101;
      marks = 85.5;
   }

 void display()
   {
      cout << "Roll Number = " << rollNo << endl;
      cout << "Marks = " << marks << endl;
   }
};

int main()
{
   Student s1;      // Default constructor called automatically

   s1.display();

   return 0;
}
```

Constructor

Parameterized constructor:

A **parameterized constructor** is a constructor that takes one or more arguments and uses those arguments to initialize the data members of an object.

Constructor

Parameterized constructor:

```
class ClassName
{
private:
   // private members

public:

   ClassName(parameter1, parameter2, ...)
   {
      // constructor body
   }
};
```

```
#include <iostream>
using namespace std;
class Point
{
    int m, n;
public:
    Point(int a, int b);        // Inline Parameterized constructor definition
{
    m = a;
    n = b;
}
   void display()
    {
        cout << "m = " << m << "\n";
        cout << "n = " << n << "\n";
    }
};
int main()
{
    Point p1(10, 100);   // invoke parameterized Constructor
    Point p2(25, 75);
    cout << "\nOBJECT1" << "\n";
    p1.display();

    cout << "\nOBJECT2" << "\n";
    p2.display();

    return 0;
}
```

Constructor

Copy constructor

A copy constructor creates a new object by initializing it with the values of an existing object of the same class.

Example:

```
class X
{
public:
   X();            // Default constructor

   X(X &obj);      // Copy constructor

   X(int a);       // Parameterized constructor
};
```

```
#include <iostream>
using namespace std;

class code
{
   int id;

public:
   code()            // Default constructor
   {
   }

   code(int a)       // Parameterized constructor
   {
      id = a;
   }

   code(code &x)     // Copy constructor
   {
      id = x.id;
   }
 void display(void)
   {
      cout << id;
   }
};

int main()
{
   code A(100);       // Object A is created and initialized

   code B(A);         // Copy constructor called

   code C = A;        // Copy constructor called again

   code D;            // D is created, not initialized

   D = A;             // Assignment; copy constructor is NOT called

   cout << "\nID of A: ";
   A.display();

   cout << "\nID of B: ";
   B.display();

   cout << "\nID of C: ";
   C.display();

   cout << "\nID of D: ";
   D.display();

   return 0;
}
```

Constructor Overloading

Constructor overloading means defining **more than one constructor in the same class**, where each constructor has the **same name as the class** but a **different parameter list (signature)**.

The difference can be based on:

- Number of parameters
- Data types of parameters
- Both number and data types

Constructor Overloading

Syntax:

```
class ClassName
{
public:

    ClassName()
    {
        // Default constructor
    }

    ClassName(dataType1 arg1, dataType2 arg2)
    {
        // Another overloaded constructor
    }
```

```
include <iostream>
using namespace std;

class complex
{
    float x, y;

public:
    // Constructor: no argument
    complex()
    {
        x = 0;
        y = 0;
    }
    // Constructor: one argument
    complex(float a)
    {
        x = a;
        y = 0;
    }

    // Constructor: two arguments
    complex(float real, float imag)
    {
        x = real;
        y = imag;
    }
```

```
 // Friend function declarations
    friend complex sum(complex, complex);
    friend void show(complex);
};

// Friend function to add two complex numbers
complex sum(complex c1, complex c2)
{
    complex c3;

    c3.x = c1.x + c2.x;
    c3.y = c1.y + c2.y;

    return c3;
}
  // Friend function to display a complex number
void show(complex c)
{
    cout << c.x << " + j" << c.y << "\n";
}

int main()
{
    complex A(2.7, 3.5);
    complex B(1.6);
    complex C;

    C = sum(A, B);
```

```
cout << "A = ";
   show(A);

   cout << "B = ";
   show(B);

   cout << "C = ";
   show(C);
// Another way to give initial values
   complex P, Q, R;

   P = complex(2.5, 3.9);
   Q = complex(1.6, 2.5);
   R = sum(P, Q);
   cout << "\n";

   cout << "P = ";
   show(P);

   cout << "Q = ";
   show(Q);

   cout << "R = ";
   show(R);

   return 0;
}
```

Friend Function

- A Friend function is a non-member function of the class that has been granted access to all private members of the class.
- We simply declare the function within the class by a prefixing its declaration with keyword friend.
- Function definition must not use keyword friend.
- Definition of friend function is specified outside the class body and is not treated as a part of the class.

Friend Function

Syntax:

Syntax: class ABC

{ ……

public:

friend void xyz(object of class);

};

Friend Function

```
#include <iostream>
using namespace std;

class Demo
{
   int x;

public:
   Demo(int a)
   {
      x = a;
   }
 friend void display(Demo);
};

void display(Demo d1)
{
   cout << d1.x;
}

int main()
{
   Demo d2(5);
   display(d2);

   return 0;
}
```

Destructor

- A destructor is a special member function of a class that is automatically invoked when an object is destroyed or goes out of scope.
- It has the same name as the class preceded by a tilde (~), takes no arguments, has no return type, and cannot be overloaded.
- Its main purpose is to perform cleanup and release resources, such as dynamically allocated memory, associated with the object.

Destructor

- Syntax:

```
class ClassName
{
public:
    ~ClassName()
    {
        // destructor body
    }
};
```

```
#include <iostream>
using namespace std;

class Test {
public:

    // User-Defined Constructor
    Test() {
        cout << "Constructor Called"
             << endl;
    }

    // User-Defined Destructor
    ~Test() {
        cout << "Destructor Called"
             << endl;
    }
};
int main() {
    Test t;

    return 0;
}
```

Dynamic Initialization through Constructor

Dynamic initialization through constructors is the process of initializing class objects at runtime by supplying values to constructors. Constructor overloading allows objects to be initialized in different formats according to the requirements of the program.

```
#include <iostream>
using namespace std;

class Person
{
    string name, surname, thirdname;

public:
    // Constructor with one argument
    Person(string n)
    {
        name = n;
    }

    // Constructor with two arguments
    Person(string n, string s)
    {
        name = n;
        surname = s;
    }

    // Constructor with three arguments
    Person(string n, string s, string t)
    {
        name = n;
        surname = s;
        thirdname = t;
    }

    void display()
    {
        cout << name << " " << surname << " " << thirdname << endl;
    }
};

int main()
{
    Person p1("Amit");
    Person p2("Amit", "Sharma");
    Person p3("Amit", "Kumar", "Sharma");

    p1.display();
    p2.display();
    p3.display();

    return 0;
}
```

Static Data Members with Constructors and Destructors

Static data members with constructors and destructors can be used to keep track of the number of active objects and to perform initialization only when the first object is created and cleanup when the last object is destroyed.

```
#include <iostream>
using namespace std;

class Demo
{
   static int count;

public:
   Demo()
   {
      count++;
      cout << "Object created. Count = " << count << endl;
   }
   ~Demo()
   {
      count--;
      cout << "Object destroyed. Count = " << count << endl;
   }
};

int Demo::count = 0;

int main()
{
   Demo d1;
   Demo d2;
   Demo d3;

   return 0;
}
```

Pointers to objects

C++ provides two special operators for **dynamic memory management**:

- **new** → creates a dynamic object and allocates memory for it.
- **delete** → releases the memory occupied by a dynamic object.

- A pointer to an object is a pointer variable that stores the address of an object of a class.
- It can be used to access the data members and member functions of that object.
- Pointers can store the addresses of primitive data types as well as objects of user-defined classes.

Syntax:

```
ClassName *pointer_name;
```

Pointers to objects

Example:

```
Student *ptr;
ptr = new Student;   // creates dynamic object

delete ptr;          // destroys object and releases memory
```

Pointers to objects

- **Declaring a pointer to an object**

```
ClassName *ptr_to_object;
```

ptr_to_object is a pointer that can point to an object of ClassName.

- **Pointing to a statically created object**

```
ptr_to_object = &object;
```

Store the address of an existing/static object in the pointer.

- **Pointing to a dynamically created object**

```
ptr_to_object = new ClassName;
```

Create an object dynamically and store its address in the pointer.

Pointers to objects

Accessing Member of object:

To access the public members using an object **Dot (.)** operator is used, and to access public members using object pointer, the **arrow (->)** operator is used.

```
pointer_to_object -> member_name
                or
(*pointer_to_object).member_name
```

Pointers to objects

```
#include <iostream>
using namespace std;

class SomeClass
{
public:
   int data1;
   char data2;

   SomeClass()
   {
      cout << "Constructor SomeClass() is invoked\n";
      data1 = 1;
      data2 = 'A';
   }

   ~SomeClass()
   {
      cout << "Destructor ~SomeClass() is invoked\n";
   }
   void show()
   {
      cout << "data1 = " << data1;
      cout << " data2 = " << data2 << endl;
   }
};

int main()
{
   SomeClass *ptr;
   SomeClass object1;

   ptr = &object1;

   cout << "Accessing object through object1.show()..." << endl;
   object1.show();

   cout << "Accessing object through ptr->show()..." << endl;
   ptr->show();

   return 0;
}
```

Pointers to objects

Dereferencing Pointers:

Dereferencing a pointer means using the pointer to access the original object or value to which it points.

Syntax:

```
DataType & ReferenceVar=* (new DataType);
```

Pointers to objects

```
#include <iostream>
#include <cstring>
using namespace std;

class Student
{
private:
   int roll_no;
   char name[20];

public:
   // Initializing data members
   void setdata(int roll_no_in, char *name_in)
   {
      roll_no = roll_no_in;
      strcpy(name, name_in);
   }
 // Display data members
   void outdata()
   {
      cout << "Roll No = " << roll_no << endl;
      cout << "Name = " << name << endl;
   }
};
```

```
int main()
{
   Student &s1 = *(new Student);    // reference to a dynamic object

   s1.setdata(1, (char*)"Savithri");
   s1.outdata();

   Student &s2 = *(new Student);    // reference to a dynamic object

   s2.setdata(2, (char*)"Bhavani");
   s2.outdata();

   Student s3;                  // normal object
   s3.setdata(3, (char*)"Vani");

   Student &s4 = s3;            // reference to existing object

   s3.outdata();
   s4.outdata();

   return 0;
}
```

Array of objects

- An object of a class represents a single record in memory. If we need to store more than one record of the same class type, instead of creating separate objects, we can create an array of objects.

- C++ allows arrays of user-defined data types, including classes.
- An array whose elements are objects of a class is called an array of objects.
- Each element of the array represents an individual object of that class.
- An array of objects is declared in the same way as an array of any other data type

Array of objects

- Syntax:

```
class class-name
{
   datatype var1;
   datatype var2;
   ...
   datatype varN;

   method1();
   method2();
   ...
   methodN();
};

class-name obj[size];
```

Array of objects

```
#include <iostream>
#include <cstring>
using namespace std;

class Student
{
private:
    int rollNo;
    char name[25];
    int age;
    float marks;
public:
    // Function to input student details
void getData()
    {
cout << "Enter Roll No: ";
        cin >> rollNo;

        cout << "Enter Name: ";
        cin >> name;

        cout << "Enter Age: ";
        cin >> age;

        cout << "Enter Marks: ";
        cin >> marks;
    }
```

Array of objects

```
// Function to display student details
    void putData()
    {
        cout << rollNo << "\t"
             << name << "\t"
             << age << "\t"
             << marks << endl;
    }
};

int main()
{
    Student s[3];      // Array of 3 Student objects
```

```
// Input records
    for (int i = 0; i < 3; i++)
    {
        cout << "\nEnter details of Student " << i +
1 << endl;
        s[i].getData();
    }
// Display records
    cout << "\nStudent Records\n";
    cout << "Roll No\tName\tAge\tMarks\n";

    for (int i = 0; i < 3; i++)
    {
        s[i].putData();
    }

    return 0;
}
```

this Pointer

- The this pointer is an implicit pointer available inside a non-static member function that contains the address of the current object.
- It points to the current object, i.e., the object that has invoked the member function.

this Pointer

```
#include <iostream>
using namespace std;

class Student {
    int age;
public:
    Student(int age) {
        this->age = age;  // data member
= parameter
    }
    void show() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    Student s(20);
    s.show();
    return 0;
}
```